Първи стъпки в C++

Школа по информатика
17 октомври 2026 г.
НП „Бъдеще за таланти“

Днешната цел

Създавам проект в Code::Blocks.
Пиша и стартирам своя C++ програма.
Въвеждам числа и пресмятам резултат.
Проверявам решението с няколко теста.

Нашите четири часа

1. Алгоритми, инсталиране и среда
2. Първа програма и извеждане
3. Променливи, вход и пресмятания
4. Самостоятелни задачи и проверка

Между часовете има почивки.

Какво прави състезателят

Прочита условието и ограниченията.
Измисля точни стъпки за решение.
Записва ги като програма.
Проверява и изпраща изходния код.
Проверката използва и непоказани тестове.

Алгоритъм

Алгоритъмът е точна последователност от стъпки за решаване на задача.
Пример: имаме две числа.
Прочитаме ги, събираме ги, записваме сбора.
Стъпките трябва да работят за всички допустими данни.

Задача без компютър

Роботът знае само: крачка напред, завой наляво, завой надясно.
Опиши как да премине по две страни на квадрат със страна 3 крачки.
Съученикът изпълнява точно инструкциите.
Коя неточна команда може да го объркa?

Език, среда и компилатор

C++ е езикът, на който записваме програмата.
Code::Blocks е средата, в която работим.
Компилаторът превежда кода до изпълнима програма.
Конзолата показва изхода и приема входа.

Инсталация за Windows

Официален адрес: codeblocks.org/downloads/binaries/
В раздел Windows избираме:
codeblocks-25.03mingw-setup.exe
Пакетът mingw включва и C++ компилатор.
В училище инсталираме с помощта на учителя.

Стъпки на инсталирането

1. Стартираме изтегления инсталатор.
2. Next, преглед на лицензa, I Agree.
3. Запазваме компонентите с MinGW.
4. Избираме папка и Install.
5. Стартираме Code::Blocks след Finish.

Първо стартиране

При откриване на компилатор избираме GNU GCC Compiler.
Задаваме го като подразбиращ се, ако има такъв избор.
Ако не е намерен: Settings > Compiler > Toolchain executables > Auto-detect.
Учителят проверява дали е инсталиран MinGW.

Ориентиране в средата

Меню File: създаване, отваряне, записване.
Projects: файловете на проекта.
Редактор: тук пишем main.cpp.
Build messages / Build log: съобщения от компилатора.
Конзола: вход и изход при изпълнение.

Нов проект

File > New > Project > Console application.
Избираме C++, после Next.
Име: FirstProgram. Папка: C:\CPP\FirstProgram.
Оставяме GNU GCC Compiler и Finish.
В Projects отваряме Sources > main.cpp.

Записване и изпълнение

Ctrl+S записва текущия файл.
Build компилира проекта.
Run стартира създадената програма.
Build and run прави и двете (обичайно F9).
След промяна отново компилираме.

Проверка след първия час

Всеки има свой проект и main.cpp.
Всеки намира редактора и Build messages.
Всеки записва файла.
Ако техниката ни забави, работим по двойки.
Следва почивка.

Нашата първа програма

Въведи кода точно. Стартирай го.
В конзолата трябва да се появи
Hello!

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello!" << '\n';
    return 0;
}
```

Какво означават редовете

iostream осигурява стандартния вход и изход.
using namespace std позволява краткото име cout.
main е началната функция.
Скобите { } ограждат нейното тяло.
return 0 означава успешно приключване.
Много инструкции завършват с ;

Текст и нов ред

Текстът стои между двойни кавички.
\n означава преминаване на нов ред.
Интервалите вътре в текста също се извеждат.

```
cout << "I learn C++" << '\n';
cout << "Group E" << '\n';
cout << "17 " << "October" << '\n';
```

Предвиди изхода

Запиши на хартия какво ще се появи.
После провери чрез изпълнение.

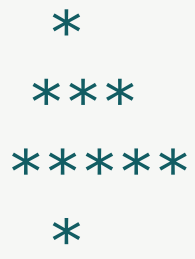
```
cout << "A";
cout << "B" << '\n';
cout << "C D" << '\n';
```

Практика: визитка

Изведи на три реда:
своето име с латински букви,
School of Informatics,
Group E.
Запази проекта като Card.
Допълнение: добави рамка от звездички.

Практика: фигура

Изведи точно следната фигура. Използвай само cout. Интервалите имат значение.



Първите грешки

Липсва ; в края на инструкция.
Липсва затваряща кавичка или скоба.
Написали сме Cout вместо cout.
В кода сме използвали кирилска буква, приличаща на латинска.
Четем първата грешка в Build messages.

Лаборатория за грешки

Намери и поправи трите проблема.
Компилирай след всяка поправка.

```
#include <iostream>
using namespace std;
int main() {
    Cout << "Hello"
    return 0;
```

Проверка след втория час

Пиша малка програма от начало.
Показвам текст на няколко реда.
Разпознавам съобщение за компилационна грешка.
Различавам програмния код от изхода.
Следва почивка.

Променлива

Променливата пази стойност,
която можем да използваме.
int пази цели числа в допустимия
за типа диапазон.
Името започва с буква или долна
черта.
Използваме ясни имена на
латиница.

```
int apples = 5;
int pears = 3;
cout << apples + pears << '\n';
```

Имена и присвояване

Подходящи имена: age, total, sideA.
Неподходящи: 2age, my age, int.
Главните и малките букви се различават.
Символът = записва нова стойност.

```
int score = 4;
score = score + 3;
cout << score; // 7
```

Вход с cin

cin чете данните от конзолата.
След въвеждане натискаме Enter.
Числата могат да са разделени с интервали или нови редове.

```
int a, b;
cin >> a >> b;
cout << a + b << '\n';
```

Пълно решение: сбор

Вход: две цели числа от 0 до 1000.
Изход: техният сбор.
Пример: вход 7 12, изход 19.

```
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    cout << a + b << '\n';
    return 0;
}
```

Точен състезателен изход

При вход 7 12 се очаква само 19.
Не добавяме „Въведи число“ или „Сборът е“.
Следваме броя на редовете и реда на резултатите.
Съобщенията на Code::Blocks за време и код на изход не са част от cout.

Аритметични операции

+ събира, - изважда, * умножава.
При цели положителни числа / дава цялата част на частното.
% дава остатъка при целочислено деление.
Делителят трябва да е различен от нула.

```
cout << 17 / 5 << '\n'; // 3
cout << 17 % 5 << '\n'; // 2
cout << 2 + 3 * 4;      // 14
```

Скоби и ред на действията

Умножението и делението са преди събирането.
Скобите променят реда.
Какво извежда всеки израз?

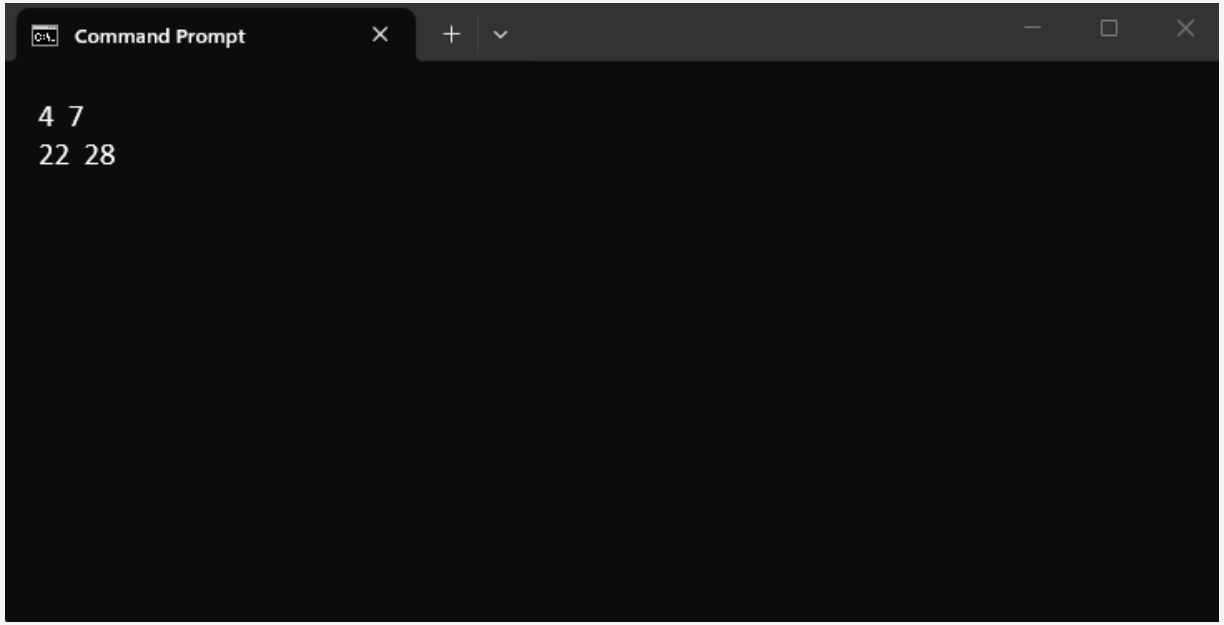
```
cout << 2 + 3 * 4 << '\n';
cout << (2 + 3) * 4 << '\n';
cout << 20 / 3 << '\n';
cout << 20 % 3 << '\n';
```

Проверка след третия час

Въвеждам две числа и извеждам резултат.
Обяснявам разликата между / и %.
Проверявам със собствени входни данни.
Задачите с деление остават за допълнение, ако входът още затруднява групата.
Следва почивка.

Задача 1: правоъгълна градина

Вход: цели страни a и b в метри, 1 ≤ a,b ≤ 1000.
Изход: периметърът и лицето, разделени с интервал.
Пример: вход 4 7, изход 22 28.
Първо запиши формулите, после програмата.
Време: 8 минути.



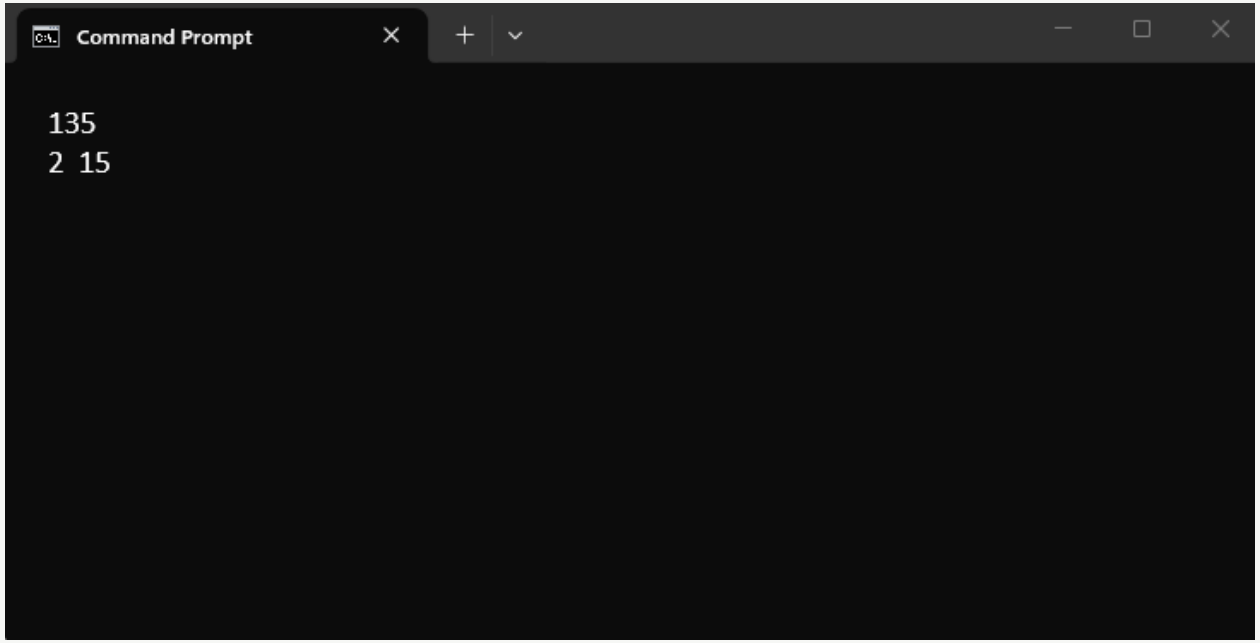
Решение: градина

Периметърът е 2*(a+b). Лицето е a*b.
Провери и квадрат със страна 1.

```
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    cout << 2 * (a + b) << " "
         << a * b << '\n';
    return 0;
}
```

Задача 2: минути

Вход: цяло число m, 0 ≤ m ≤ 10000, общ брой минути.
Изход: цели часове и оставащи минути.
Пример: вход 135, изход 2 15.
Провери 0, 59, 60 и 61.
Време: 8 минути.



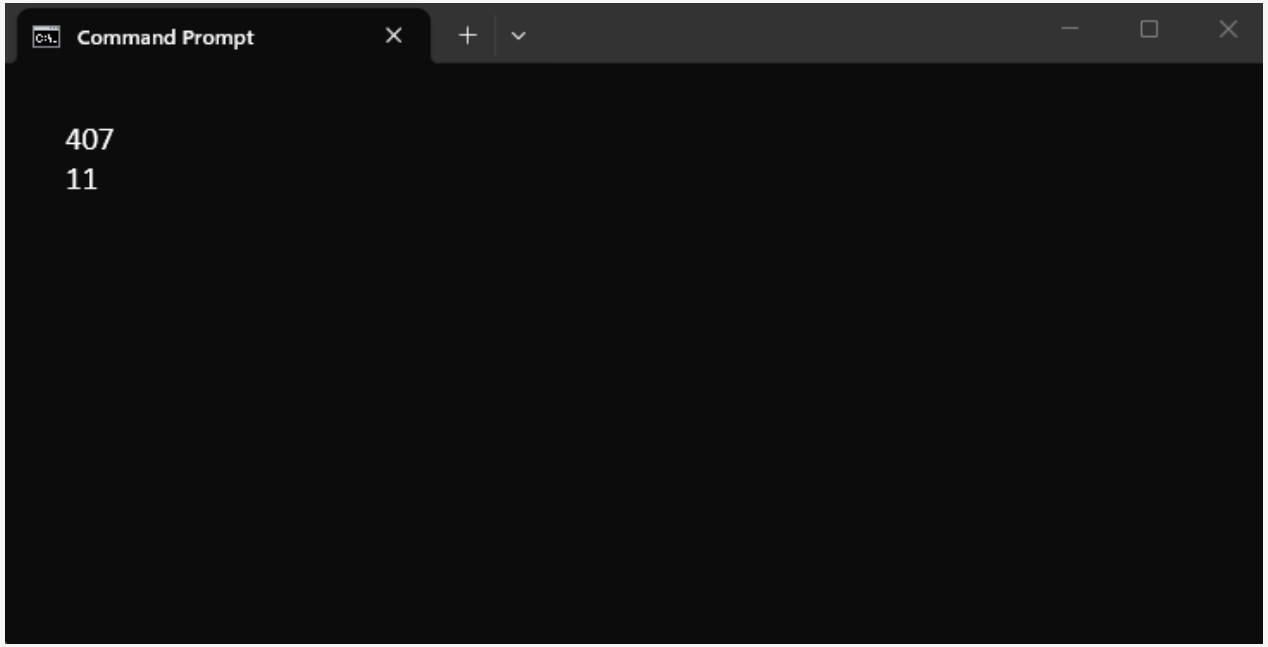
Решение: минути

В един час има 60 минути.
Целите часове са m/60, остатъкът е m%60.

```
#include <iostream>
using namespace std;
int main() {
    int m;
    cin >> m;
    cout << m / 60 << " "
         << m % 60 << '\n';
    return 0;
}
```

Задача 3: цифри, за напредващи

Вход: трицифрено естествено число n.
Изход: сборът на цифрите му.
Пример: вход 407, изход 11.
Подсказка: n/100, (n/10)%10, n%10.
Провери 100, 909 и 999.



Решение: цифри

За 407 получаваме 4, 0 и 7.
Събираме ги.

```
int n;
cin >> n;
int hundreds = n / 100;
int tens = (n / 10) % 10;
int ones = n % 10;
cout << hundreds + tens + ones;
```

Тестове, които откриват грешки

Примерът показва само един случай.
Проверяваме най-малките разрешени стойности.
Проверяваме граница: 59, 60, 61 минути.
При цифри проверяваме нула в средата: 407.
Изчисляваме очаквания резултат предварително.

Изпращане на решение

Учителят показва избраната проверяваща система.
Отваряме правилната задача и език C++.
Изпращаме изходния код от main.cpp според указанията.
Четем резултата и поправяме решението.
Конкретните бутони зависят от системата.

Какво означава резултатът

Accepted: решението преминава тестовете на системата.
Wrong Answer: има грешен резултат или формат.
Compilation Error: кодът не се компилира.
Time Limit: изпълнението е прекалено бавно.
Един успешен пример още не доказва вярно решение.

Изходен билет

1. Създай програма за сбор на три цели числа.
2. Провери с 0 0 0 и 2 5 9.
3. Предай main.cpp и запиши резултатите.
4. Обясни защо 7/2 е 3 при целочислено деление.

Допълнение: покажи и остатъка.

Домашни задачи

Д1. Визитка на три реда и фигура по избор.
Д2. Две цели числа от 0 до 1000: сбор и произведение.
Д3. Куб със страна 1..100: обем.
Д4. Секунди 0..100000: минути и оставащи секунди.
Д5. Трицифрено число: цифрите в обратен ред, като отделни числа.

Самоподготовка през седмицата

Ден 1: повтори първата програма без преписване.
Ден 2: реши Д1 и Д2.
Ден 3: реши Д3 и провери границите.
Ден 4: опитай Д4 и Д5.
Пази дневник: грешка, причина, поправка.
Донеси един въпрос за следващото занятие.

Следващите срещи

24 октомври: числови изрази, цифри и първи условия.
31 октомври: есенна ваканция, без занятие.
7 и 14 ноември: условия, знаци и комбинирани задачи.
21 ноември: пробна тема и анализ.
Есенният турнир е първа възможност за опит според готовността.

Справочник за следващото занятие

int е удобен за малки цели стойности.
long long използваме за по-големи цели резултати.
double пази приблизителни реални стойности.
7/2 е 3, а 7.0/2 е 3.5.
Типовете и изборът според ограниченията ще упражняваме подробно.

Полезни адреси

Code::Blocks: codeblocks.org/downloads/binaries/
Календар и архив: infos.infosbg.com
Учебна програма за група Е: раздел „Документи“, 2025/2026.
Използваме я за методическа основа.
Условията за участие се проверяват по актуалния регламент.